

## Model Stealing: Stolen Model Detection

---

### Task: Find out which image classification models were stolen from the target model

Training machine learning models, especially foundation models, comes with huge costs, both computational and monetary. This makes it attractive for attackers to extract or steal these models by less costly means, and use them for malicious purposes. Thus, it is important to be able to detect these stolen models. In this task, you are in the victim's shoes, and your goal is to detect which models among a set of suspect image classification models are stolen from your target model, either verbatim or with changes applied to the model.

#### As the victim, you get:

- White-box access to the target model weights, along with information about how it was trained, including the training dataset, used augmentations, etc.
- White-box access to 360 suspect model weights (All available on [HuggingFace](https://huggingface.co))

#### What you don't have access to:

- The exact training procedures, training histories, and provenance labels of the suspect models.

#### What's your task:

Your goal is to determine which models are directly stolen, or at least derived from the target model. This is indicated through a stealing confidence score. We expect you to submit **continuous stealing confidence scores** (you are *not* supposed to submit scores as 0 (genuine) and 1 (stolen)).

**Note** - The stealing confidence score reflects how likely it is that a given model was stolen or derived from the target model. Thus, a higher score means that the model is likely to be stolen.

Concretely, the set of potentially stolen models includes models that:

- are fully independent and trained from scratch, either on a different dataset, a different subset, or under a different training recipe than the target model (counts as **not stolen**)
- are independent but trained on the same underlying data distribution, possibly even overlapping substantially with the target training data (counts as **not stolen**)
- are direct copies of the target model (counts as **stolen**)
- are derived from the target model through post-processing or function-preserving transformations (counts as **stolen**)
- originate from the target model or one of its intermediate checkpoints and are then further trained or partially finetuned (counts as **stolen**)

- are obtained through model extraction or distillation, for example by training a student model on the target model's outputs, including with out-of-domain transfer data (counts as **stolen**)

A model is considered stolen if it is directly copied or derived from the target model, even if substantial modifications were applied afterwards.

### Evaluation Metric

Submissions are evaluated using:

$$Score = TPR@5\%FPR$$

Where:

- TPR (True Positive Rate): fraction of stolen model correctly identified
- FPR (False Positive Rate): fraction of stolen model incorrectly identified

Higher values indicate better detection performance.

### Target Model Details

The target model is a CIFAR-style ResNet-18 classifier for 100 classes. The model was trained from scratch on a subset of the CIFAR-100 training split. The exact indices of the training samples are provided in [train\\_main\\_idx.json](#).

During training, the following augmentations were used:

- Random horizontal flipping
- Biased random cropping with reflection padding of 4 pixels

For the biased crop, the parameters were:

- `bias_x = 0.5`
- `bias_y = -0.25`
- `jitter = 0.25`

After augmentation, images were normalized using

`mean = (0.5071, 0.4867, 0.4408)` and  
`std = (0.2675, 0.2565, 0.2761).`

The model was trained using cross-entropy loss with no label smoothing, SGD with momentum 0.9, weight decay 5e-4, batch size 256, and an initial learning rate of 0.1 with a cosine annealing learning-rate schedule.

### Your Starting Point

To start you off, we give you a [task\\_template.py](#) that shows you how to load the models and save submissions while [submission.py](#) instructions explaining the submission process and code.

### How to submit your results?

- We are provided a unique API-key to every participating team. You are supposed to only use this token to submit your results to our server.
- Your submissions should consist of a .csv file with the columns “id” for the model id with values from 0 to 359, and “score” for the stealing confidence score for that particular model.
- You are supposed to submit your files using the code provided in the coding template. Remember to replace `YOUR_API_KEY_HERE` with your actual API-key (keep the double quotes), and replace `PATH/TO/YOUR/SUBMISSION.csv` with your real results. Also set `SUBMIT` to True if not already.
- To avoid the possibility of brute-forcing, only one submission can be made every 60 minutes. If the submission was unsuccessful due to an error, this cooldown period will only be 2 minutes long.

### Submission Format

The submission file must be named `submission.csv` with an `id` and a `score` field:

```
id,score
0,0.82
1,0.11
2,0.32
```

### Scoring

All submissions are automatically validated and evaluated on the hidden test set.

- **Public leaderboard:** based on 30% of models
- **Private leaderboard:** final ranking based on the remaining 70%

This means that **optimizing only the leaderboard score does not automatically give you a good final result**. Your results must generalize for all models. The leaderboard score reflects the `score` achieved by the submission. Higher is better.

### Leaderboard

Results are published at: [http://34.63.153.158/leaderboard\\_page](http://34.63.153.158/leaderboard_page)

The leaderboard shows the best result per team only. If your current submission score is lower than the score saved in the leaderboard, the score will not be updated.

### Points to note regarding submission:

You can only submit a CSV with scores **every 60 minutes**. If your submission is malformed or wrong, you should get a comprehensive error message. However, even if you fail, you get a **2 minute** cooldown.

### Read the following instructions carefully:

Additionally, you are required to submit a **ZIP file** to CMS that contains your code and a report that explains your approach to the task. This file should contain:

1. Report
  - Matriculation number and CMS team id in the report
  - An Introduction:

- A short paragraph explaining **your understanding of the task** written by your own words.
- A Main body:
  - Explaining **your approach to the task** including the details about the hyperparameters and structural choices you have made, as well as the reasons you made that choice.
  - Explaining **key results** that you have obtained on the public leaderboard.
- A Conclusion
  - In your conclusion try to be goal oriented. In other words, try to answer this question: now that we have successfully detected the stolen models, what practical implications does stealing some model may have and why should we be concerned about it?

## 2. Code Repository

- Containing all your code files, well documented and clean.
- Avoid including the large models and artifacts in your repository.
- Add a README.md to your code repository that **ONLY** explains how to **recreate your best result** that appears on the leaderboard.
- You are only supposed to make one submission per group.

The report should NOT be more than **2 pages** long. The submitted zip file that does not contain the code and/or the report will NOT be considered for evaluation. For the report you SHOULD use the [template for ICLR](#) on overleaf. Note that there is no need for an abstract and you should skip it.

The submission to the leaderboard will end automatically on **26.05.2026 23:59** and after that you will not be able to submit any results to the leaderboard. The same date also applies to CMS. As a result, make sure to submit your results and report on time. Thereafter, you will be able to check your evaluation results performed on the held-out set on the leaderboard. No late submissions will be accepted via email (no exceptions).

## References

- "High Accuracy and High Fidelity Extraction of Neural Networks" Matthew Jagielski, Nicholas Carlini, David Berthelot, Alex Kurakin, Nicolas Papernot. USENIX Security 2020. <https://arxiv.org/abs/1909.01838> (Model Stealing Background)
- "Knockoff Nets: Stealing Functionality of Black-Box Models" Tribhuvanesh Orekondy, Bernt Schiele, Mario Fritz. CVPR 2019. <https://arxiv.org/pdf/1812.02766> (Model Stealing Technique)
- "ModelDiff: A Framework for Comparing Learning Algorithms" Harshay Shah, Sung Min Park, Andrew Ilyas, Aleksander Madry. ICML 2023. <https://arxiv.org/pdf/2211.12491> (Method for comparing model similarity)